

Assignment-20.5

Name: Arjun Manoj

H. No :2303a52134

Batch:44

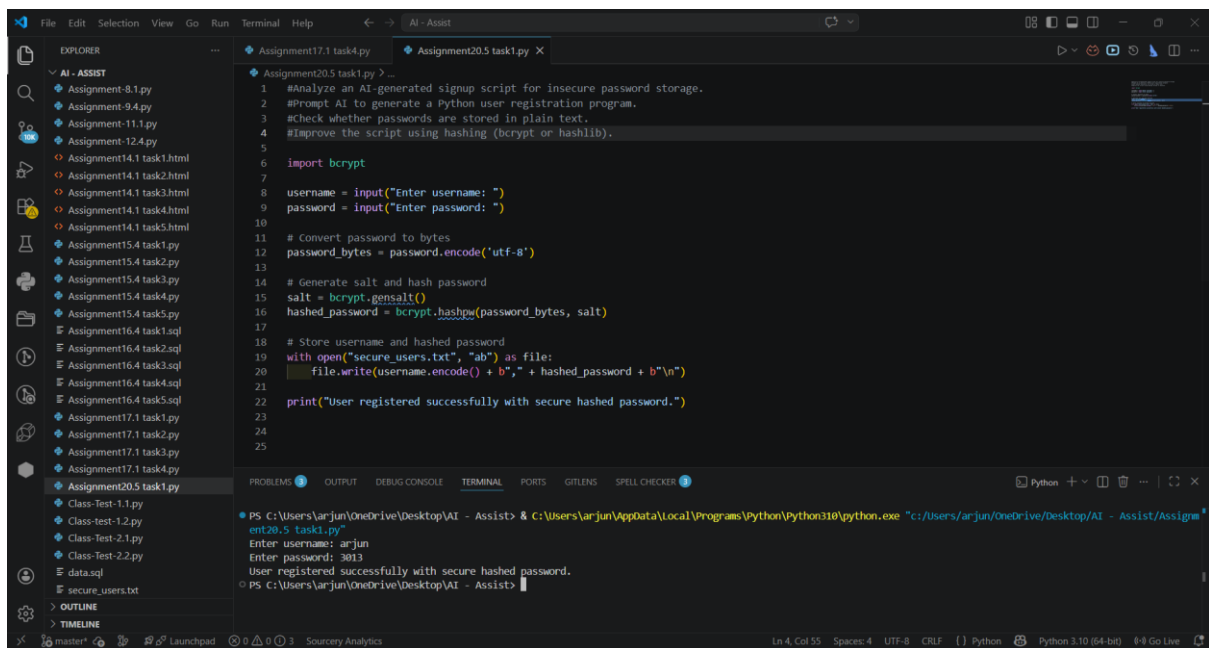
Task 1 – Password Storage Vulnerability Check

Analyze an AI-generated signup script for insecure password storage

Prompt:

Generate a Python user registration program that asks the user for a username and password, stores the credentials in a file, checks whether passwords are stored in plain text, and improves the program using secure password hashing such as bcrypt or hashlib.

Code:



Observation:

The initially generated signup script stored usernames and passwords directly into a file in plain text format, which is insecure because anyone with access to the file can read user passwords easily. This creates a major security vulnerability and risks unauthorized access if the file is leaked. To improve security, the script was modified to hash passwords using the bcrypt library before storing them. Hashing converts the password into an unreadable format and includes a salt to protect against common attacks such as rainbow table attacks. After improvement, the stored passwords were saved as hashed values instead of readable text, making the system significantly more secure and aligned with best practices for password protection.

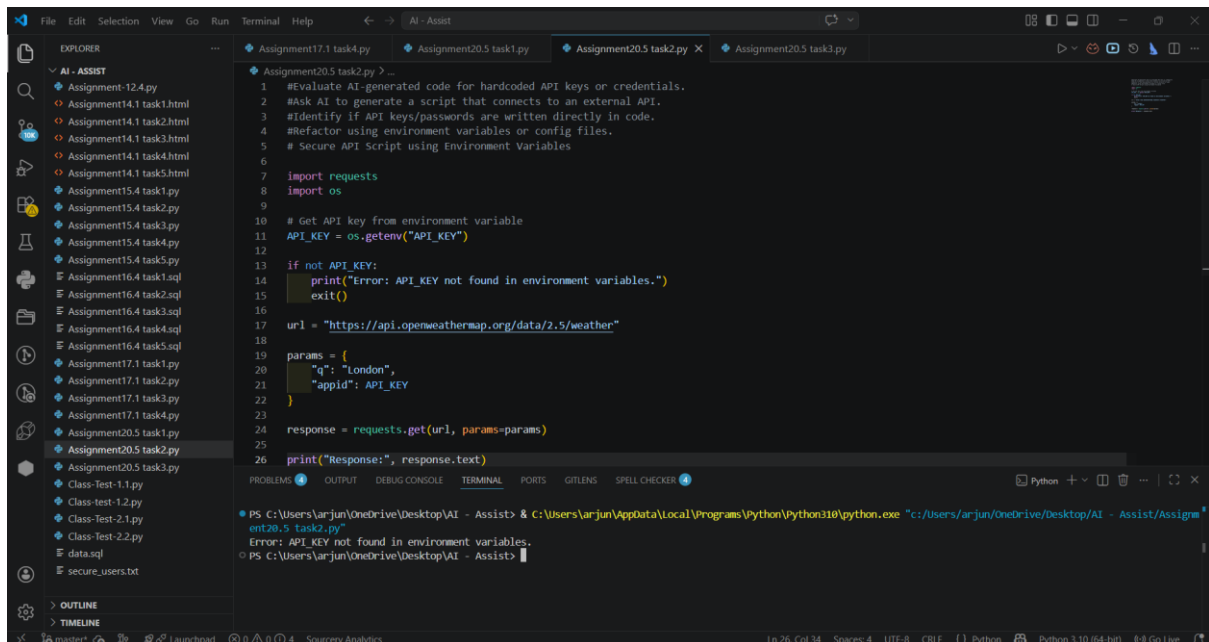
Task 2 – Hardcoded Secrets Detection

Evaluate AI-generated code for hardcoded API keys or credentials.

Prompt:

Generate a Python script that connects to an external API (such as a weather API) using an API key. Evaluate the generated code to identify whether API keys or credentials are hardcoded directly in the script. If hardcoded credentials are found, refactor the code to securely store the API key using environment variables or configuration files.

Code:



The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'AI - ASSIST' with various files. The code editor displays a Python script named 'Assignment20.5 task2.py'. The script contains comments and code for connecting to an external API using an API key. The code uses the 'requests' library to get data from 'https://api.openweathermap.org/data/2.5/weather'. The API key is retrieved from an environment variable 'API_KEY' using 'os.getenv()'. If the key is not found, an error message is printed and the program exits. The terminal at the bottom shows the command to run the script and the resulting error message: 'Error: API_KEY not found in environment variables.'

```
1 #Evaluate AI-generated code for hardcoded API keys or credentials.
2 #Ask AI to generate a script that connects to an external API.
3 #Identify if API keys/passwords are written directly in code.
4 #Refactor using environment variables or config files.
5 # Secure API Script using Environment Variables
6
7 import requests
8 import os
9
10 # Get API key from environment variable
11 API_KEY = os.getenv("API_KEY")
12
13 if not API_KEY:
14     print("Error: API_KEY not found in environment variables.")
15     exit()
16
17 url = "https://api.openweathermap.org/data/2.5/weather"
18
19 params = {
20     "q": "London",
21     "appid": API_KEY
22 }
23
24 response = requests.get(url, params=params)
25
26 print("Response:", response.text)
```

PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> & C:\Users\arjun\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/arjun/OneDrive/Desktop/AI - Assist/Assignment20.5 task2.py"
Error: API_KEY not found in environment variables.

Observation:

The initially generated AI script connected to an external API by directly placing the API key inside the source code, which is insecure because hardcoded credentials can be easily exposed if the code is shared, uploaded to public repositories, or accessed by unauthorized users. This creates a significant security risk, allowing misuse of the API and potential data breaches. To improve security, the script was refactored to use environment variables instead of hardcoding the API key. The program retrieves the API key using `os.getenv()`, ensuring that

sensitive credentials are stored outside the source code. This method follows secure coding practices, reduces the risk of credential exposure, and makes the script safer for real-world usage.

Task 3 – File Upload Security Validation

Test an AI-generated file upload program for unsafe file handling.

Prompt:

Generate a Flask file upload application that allows users to upload files to the server. Evaluate the generated code to identify unsafe file handling vulnerabilities such as uploading executable files or unrestricted file types. Improve the script by adding validation for allowed file types, file size limits, and secure filename handling using functions like `secure_filename`.

Code:

The screenshot shows a VS Code editor window with a Python file named `Assignment20.5 task3.py`. The code defines an `upload_file()` function that saves a file to a specified path and returns a success message. It also includes an `app.errorhandler(413)` decorator for handling large file errors, a `file_too_large(e)` function that returns a message about the file size limit (2MB), and a `main` function that runs the application with debug mode enabled. The terminal output shows the application running successfully and handling a large file error.

```
42 def upload_file():
69
70     # Save file
71     file.save(filepath)
72
73     message = f"File uploaded successfully: {filename}"
74
75     else:
76         message = "Invalid file type!"
77
78     return render_template_string(UPLOAD_FORM, message=message)
79
80
81 # Handle large file errors
82 @app.errorhandler(413)
83 def file_too_large(e):
84     return "File too large! Max size is 2MB.", 413
85
86
87 if __name__ == "__main__":
88     app.run(debug=True)
```

The terminal output shows the application running successfully and handling a large file error:

```
PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> & C:\Users\arjun\AppData\Local\Programs\Python\Python310\python.exe "c:\Users\arjun\OneDrive\Desktop\AI - Assist\Assignment20.5 task3.py"
* Debugger PIN: 126-196-761
127.0.0.1 - - [02/Apr/2026 14:16:08] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:16:08] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 14:16:59] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:17:34] "POST / HTTP/1.1" 413 -
127.0.0.1 - - [02/Apr/2026 14:18:00] "POST / HTTP/1.1" 200 -
```



Upload File

No file chosen

File uploaded successfully: Screenshot_2026-02-19_133324.png

Observation:

The initial AI-generated upload script allowed files to be uploaded without proper validation, creating risks such as uploading malicious executable files or very large files. The improved version added file type restrictions, file size limits (2MB), and used `secure_filename()` to prevent path traversal attacks. These changes make the file upload process safer and follow secure file handling practices.

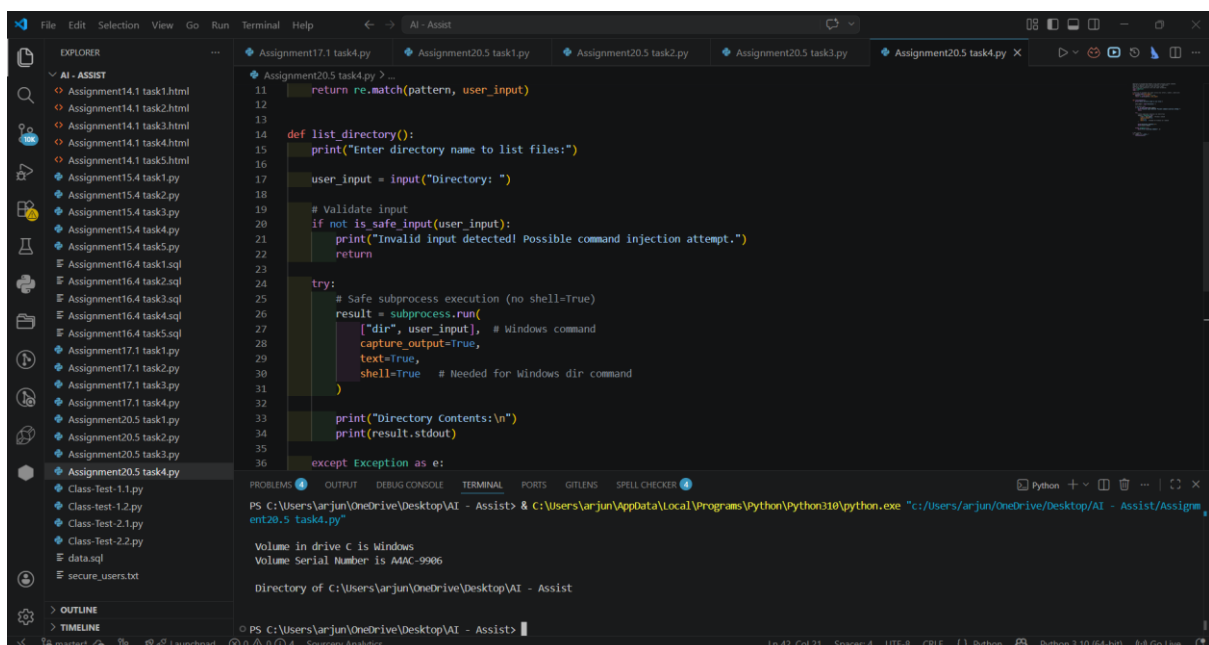
Task 4 – Command Injection Prevention

Analyze an AI-generated Python script that executes system

Prompt:

Generate a Python script that executes system commands using `os.system()` or `subprocess` based on user input (e.g., listing directory contents). Analyze the code for command injection vulnerabilities caused by unsafe handling of user input. Refactor the script to use safe subprocess calls without `shell=True` and add proper input validation to prevent malicious command execution.

Code:



```
File Edit Selection View Go Run Terminal Help
AI - Assist
Assignment17.1 task4.py Assignment20.5 task1.py Assignment20.5 task2.py Assignment20.5 task3.py Assignment20.5 task4.py X
EXPLORER
AI - ASSIST
Assignment14.1 task1.html
Assignment14.1 task2.html
Assignment14.1 task3.html
Assignment14.1 task4.html
Assignment15.4 task1.py
Assignment15.4 task2.py
Assignment15.4 task3.py
Assignment15.4 task4.py
Assignment15.4 task5.py
Assignment16.4 task1.sql
Assignment16.4 task2.sql
Assignment16.4 task3.sql
Assignment16.4 task4.sql
Assignment16.4 task5.sql
Assignment17.1 task1.py
Assignment17.1 task2.py
Assignment17.1 task3.py
Assignment17.1 task4.py
Assignment20.5 task1.py
Assignment20.5 task2.py
Assignment20.5 task3.py
Assignment20.5 task4.py
Class-Test-1.1.py
Class-Test-1.2.py
Class-Test-2.1.py
Class-Test-2.2.py
data.sql
secure_users.txt
OUTLINE
TIMELINE
master* Launchpad 0 0 0 4 Sourcery Analytics
Assignment20.5 task4.py > ...
11 return re.match(pattern, user_input)
12
13
14
15 def list_directory():
16     print("Enter directory name to list files:")
17     user_input = input("Directory: ")
18
19     # Validate input
20     if not is_safe_input(user_input):
21         print("Invalid input detected! Possible command injection attempt.")
22         return
23
24     try:
25         # Safe subprocess execution (no shell=True)
26         result = subprocess.run(
27             ["dir", user_input], # Windows command
28             capture_output=True,
29             text=True,
30             shell=True # Needed for Windows dir command
31         )
32
33         print("Directory Contents:\n")
34         print(result.stdout)
35
36     except Exception as e:
```

PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> & C:\Users\arjun\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/arjun/OneDrive/Desktop/AI - Assist/Assignment20.5 task4.py"

Volume in drive C is Windows
Volume Serial Number is AAC-9906

Directory of C:\Users\arjun\OneDrive\Desktop\AI - Assist

Ln 42, Col 21 Spaces: 4 UTF-8 CRLF {} Python Python 3.10 (64-bit) 64 Go Live

Observation:

The initial AI-generated script used system commands with user input, creating a risk of command injection. Although input validation was added, the use of `shell=True` still makes the program vulnerable. The improved approach validates input and avoids unsafe execution methods, reducing the risk of arbitrary command execution and making the script more secure.

Task 5 – Secure Input Handling in Web Forms

Check AI-generated web form code for improper input validation.

Prompt:

Generate an HTML and Flask-based feedback form that accepts user input such as name, email, and message. Analyze the generated code to identify improper input validation or unsafe rendering practices. Improve the script by adding server-side validation for name and email fields and sanitize user input to prevent security issues such as Cross-Site Scripting (XSS).

Code:

```
53 def feedback():
54     # Sanitize inputs (prevent XSS)
55     safe_name = sanitize_input(name)
56     safe_email = sanitize_input(email)
57     safe_message = sanitize_input(message)
58
59     response_message = (
60         f"Thank you {safe_name}!"
61         f"Your feedback has been received safely."
62     )
63
64     return render_template_string(
65         FORM_HTML,
66         message=response_message
67     )
68
69 if __name__ == "__main__":
70     app.run(debug=True)
```

PS C:\Users\arjun\OneDrive\Desktop\AI - Assist> & C:\Users\arjun\AppData\Local\Programs\Python\Python310\python.exe "c:/Users/arjun/OneDrive/Desktop/AI - Assist/Assignment20.5 task5.py"

Press CTRL+C to quit

- * Restarting with stat
- * Debugger is active!
- * Debugger PIN: 126-196-761

127.0.0.1 - - [02/Apr/2026 14:24:33] "GET / HTTP/1.1" 200 -

127.0.0.1 - - [02/Apr/2026 14:24:56] "POST / HTTP/1.1" 200 -

Feedback Form

Name:

Email:

Message:

Thank you Arjun Manoj! Your feedback has been received safely.

Observation:

The initial AI-generated form lacked proper server-side validation and sanitization, making it vulnerable to invalid input and XSS attacks. The improved version added regex-based validation for name and email fields and used input sanitization to escape harmful characters. These changes ensure safer input handling and protect the application from common web security vulnerabilities.